

Proceso de simulación de Péndulo

Simulacion...ulo Fase 1

 PDF



en base a este documento generar un prompts, para una función o script en lenguaje python que calcule la posición de un péndulo simple usando la fórmula del péndulo que se comporte como oscilador armónico simple, además usando la aproximación de ángulos...

Python



```
import numpy as np
import warnings

def calcular_posicion_angular(theta_max, L, t, g=9.81):
    """
    Calcula la posición angular de un péndulo simple usando la aproximación
    de ángulos pequeños (Movimiento Armónico Simple).

    Fórmula:  $\theta(t) = \theta_{\max} * \cos(\sqrt{g/L} * t)$ 

    Parámetros:
    theta_max (float): Amplitud inicial en radianes.
    L (float): Longitud del hilo en metros.
    t (float o np.array): Instante(s) de tiempo en segundos.
    g (float): Aceleración de la gravedad (default 9.81 m/s^2).

    Retorna:
    theta (float o np.array): Posición angular en radianes.
    """

    # 1. Validación de Ángulo Pequeño [cite: 38]
    # 15 grados es aprox 0.2618 radianes
    limite_angulo = np.radians(15)
    if abs(theta_max) > limite_angulo:
        warnings.warn(f"Advertencia: El ángulo inicial ({theta_max:.3f} rad) excede el límite de ángulos pequeños (~0.26 rad). El modelo lineal perderá precisión.")

    # 2. Cálculo de la Frecuencia Angular Natural (w) [cite: 22, 48]
    w = np.sqrt(g / L)
    omega = np.sqrt(g / L)
```



ahora quiero que en la simulacion sean 15 pendulos
calculando las longitudes usando una distribucion lineal
linspace de 1 a 2, en lugar de frecuencias armónicas complejas

Python



```
import numpy as np
import matplotlib.pyplot as plt

def simular_pendulos_lineales():
    # 1. Configuración de Parámetros
    n_pendulos = 15
    g = 9.81
    theta_max_deg = 10 # Grados
    theta_max_rad = np.radians(theta_max_deg) # Conversión a radianes

    # 2. Generación de Longitudes (LINEAL - linspace)
    # Solicitud: distribución lineal de 1 a 2 metros
    L = np.linspace(1, 2, n_pendulos)

    # 3. Configuración del Tiempo
    t = np.linspace(0, 60, 1000) # 60 segundos de simulación

    # 4. Cálculo Vectorizado de la Posición Angular
    # theta(t) = theta_max * cos(sqrt(g/L) * t)
    # Usamos broadcasting de NumPy: L tiene forma (15,), t tiene forma (1000,)
    # Reshape para multiplicar correctamente: (15, 1) x (1, 1000)
    omega = np.sqrt(g / L[:, np.newaxis]) # Frecuencias angulares (15, 1)
    theta = theta_max_rad * np.cos(omega * t[np.newaxis, :]) # Matriz (15, 1000)

    # 5. Visualización con Matplotlib
    plt.figure(figsize=(12, 6))

    # Graficamos cada péndulo. Usamos un mapa de colores para distinguirlos
    colors = plt.cm.viridis(np.linspace(0, 1, n_pendulos))
```

ahora convierte esta grafica estatica en una animacion que
recorra el tiempo t, el codigo lo usare en google colab, asi que
usa la libreria iptho.display



Ver razonamiento

Este código sigue los pasos de "Preparación", "Función Update" y "Visualización HTML" descritos en el texto fuente. [🔗](#)

Python



```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML

# --- 1. Configuración Física y Matemática ---
n_pendulos = 15
g = 9.81
theta_max = np.radians(10) # 10 grados (ángulos pequeños)
t_max = 20 # Duración de la animación en segundos
fps = 30 # Frames por segundo
dt = 1/fps # Paso de tiempo

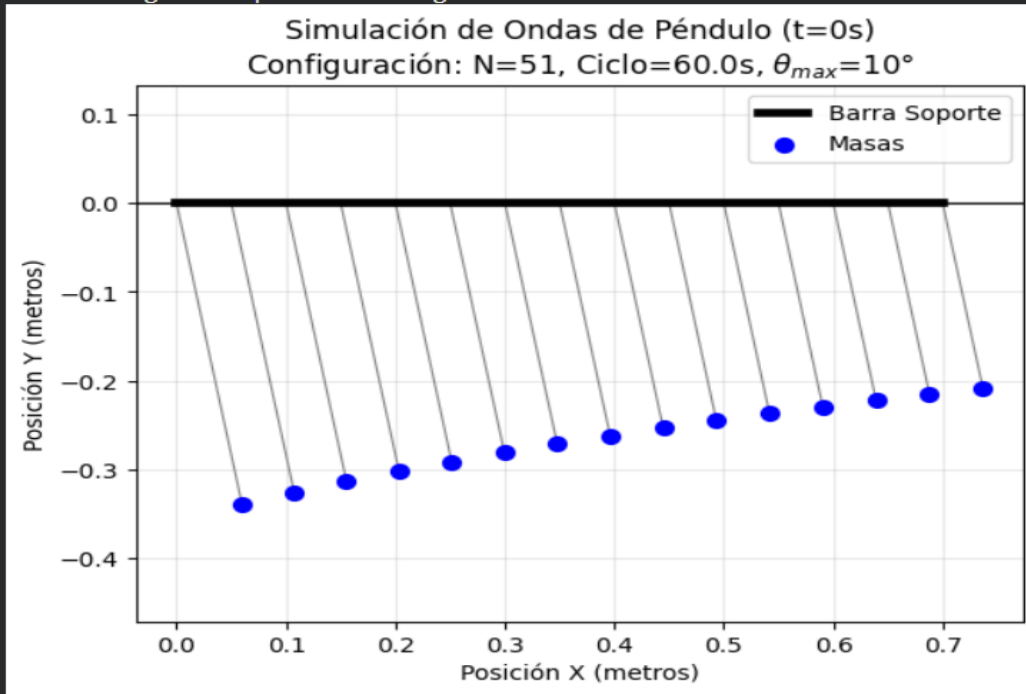
# Generación de longitudes (Lineal de 1 a 2 metros)
L = np.linspace(1, 2, n_pendulos)

# Pre-cálculo de frecuencias angulares
# omega = sqrt(g / L)
omegas = np.sqrt(g / L)

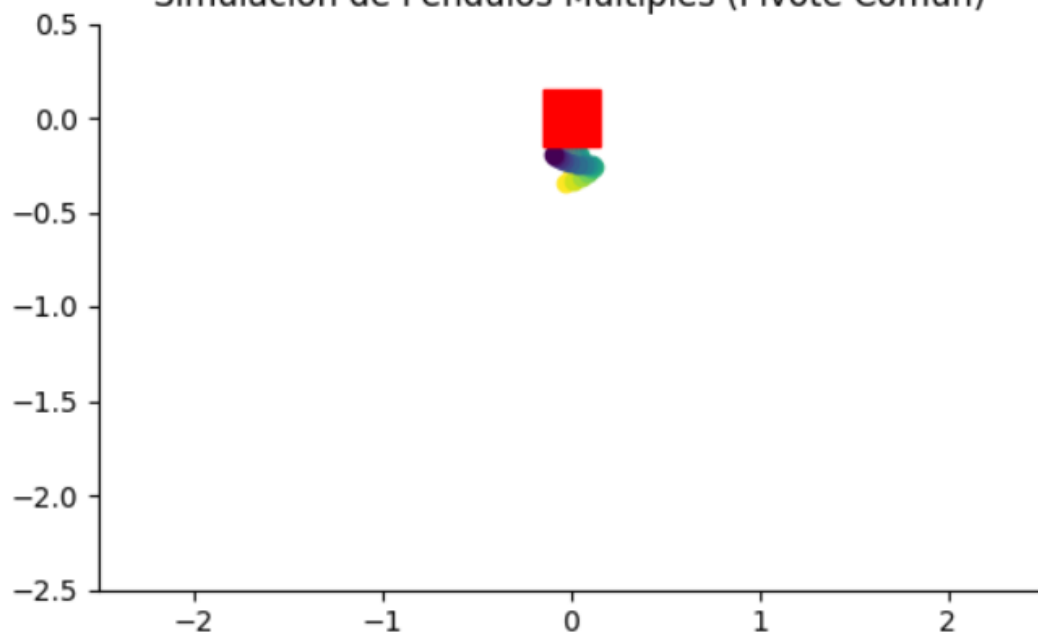
# --- 2. Preparación de la Figura (Matplotlib) ---
# [cite: 132] Configurar figura y ejes
fig, ax = plt.subplots(figsize=(8, 8))
```

[🔗 Introduce una petición para Gemini](#)

Generando gráfica para $t = 0$ segundos...



Simulación de Péndulos Múltiples (Pivote Común)



Simulacion...ulo Fase 1

PDF



en base a este documento generar un promts, para una funcion o scrpit en lenguaje phyton que calcule la posicion de un pendulo simple usando la formjula del pendulo que se comporte como oscilador armonico simple, ademas usando la aproximacion de angulos...

Python



```
import numpy as np
import matplotlib.pyplot as plt

def calcular_posicion_pendolo(theta_max, L, t, g=9.81):
    """
    Calcula la posición angular de un péndulo simple bajo la aproximación
    de ángulos pequeños (Oscilador Armónico Simple).

    Fórmula basada en el documento 'Simulación Péndulo Fase 1', Ecuación (1):
    theta(t) = theta_max * cos(sqrt(g / L) * t)

    Parámetros:
    -----
    theta_max : float
        Amplitud o desplazamiento angular máximo inicial (en radianes).
    L : float
        Longitud del péndulo (en metros).
```



ahora quiero que en la simulacion sean 15 pendulos calculnado las longitudes usando una distribucion lineal linscape de 1 a 2, en lugar de fercuencias armonicas complejas

Python



```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML

# --- 1. CONFIGURACIÓN FÍSICA ---
g = 9.81 # Gravedad (m/s^2) [cite: 23]
theta_max = np.radians(10) # Ángulo inicial pequeño (10 grados) [cite: 20, 38]
num_pendulos = 15 # Cantidad solicitada
t_max = 30 # Duración de la simulación
dt = 0.05 # Paso de tiempo

# --- 2. CÁLCULO DE LONGITUDES (DISTRIBUCIÓN LINEAL) ---
# Se usa np.linspace de 1 a 2 metros como solicitaste,
# ignorando la fórmula de "Onda de Péndulo" del documento.
```



ahora conierte esta grafica estatica en una animacion que recorra el tiempo t, el codigo lo usare en google colab, asi que usa la libreria iptho.display, para la animación utiliza el metodo `to_jshtml()` de la libreria matplotlib

Python



```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML

# --- 1. CONFIGURACIÓN DE PARÁMETROS FÍSICOS ---
g = 9.81 # Gravedad (m/s^2) [cite: 23]
theta_max_deg = 10 # Amplitud en grados (<15° para cumplir aprox. de ángulo pe
theta_max = np.radians(theta_max_deg)
num_pendulos = 15 # Cantidad de péndulos
t_max = 20 # Tiempo total de simulación (s)
fps = 30 # Frames por segundo
dt = 1 / fps # Paso del tiempo
```

Simulación de Péndulos Múltiples

